

## Week -2          Service and Category Management System

### AURA REALM

#### Online Beauty & Cosmetics Shopping Platform

#### Scenario

Glow & Grace wants to manage information about the beauty products it sells and the categories those products fall under. Each product belongs to a specific category (e.g., Skincare, Makeup, Haircare, Fragrance), and the system should maintain product details such as name, price, stock quantity, and brand. This manages the core inventory structure for the e-commerce store.

#### 1.Design Service and Category Tables

##### Category Table:

```
Connected.
SQL> CREATE TABLE Category (
2     Category_ID VARCHAR2(7),
3     Category_Name VARCHAR2(30),
4     Description VARCHAR2(100)
5 );

Table created.
```

##### Product Table:

```
SQL> CREATE TABLE Product (
2     Product_ID VARCHAR2(7),
3     Product_Name VARCHAR2(50),
4     Category_ID VARCHAR2(7),
5     Price NUMBER(8,2),
6     Stock_Quantity NUMBER,
7     Brand VARCHAR2(30)
8 );

Table created.
```

The Category and Product tables have now been created.

Next, we need to identify the relationship between the tables and apply the required constraints.

## 2. Define Relationships and Constraints

Relationships:

- A single Category can contain multiple Products.
- The relationship between Category and Product is One to Many ( $1 \rightarrow *$ ).

### KEY Constraints Explanation

PRIMARY KEY: The column or field that uniquely identifies a row or tuple.

- In the Category Table, Category\_ID is a unique value used to identify each category.
- Similarly, in the Product Table, Product\_ID is a unique column.

FOREIGN KEY: The key that connects two tables.

- Therefore, Category\_ID acts as a Foreign Key in the Product Table.

We can also consider other constraints such as NOT NULL, CHECK, and UNIQUE:

- NOT NULL: Ensures that a column cannot contain an empty (null) value. For example, a Product name or Category name cannot be empty.
- CHECK: Checks that the values in a column meet a specified condition. For example, the Price of a Product should always be a positive number, and Stock\_Quantity cannot be negative.
- UNIQUE: Ensures that a column does not contain repeated values. For example, if we apply a UNIQUE constraint to Category\_Name, no two categories can have the same name.

## Defining Relationship and Creating Constraints for Product and Category

**Primary Key:**

```
SQL> -- Primary Key Constraints
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);

Table altered.

SQL>
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);

Table altered.
```

### Creating Relationship using FOREIGN KEY:

```
SQL>
SQL> -- Foreign Key Constraint
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

### Adding Constraints:

#### NOT NULL:

```
SQL> -- NOT NULL Constraints
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR2(30) NOT NULL;

Table altered.

SQL>
SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR2(50) NOT NULL;

Table altered.
```

#### CHECK:

```
SQL> -- CHECK Constraints
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Product_Price CHECK (Price > 0);

Table altered.

SQL>
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

## UNIQUE:

```
SQL> -- UNIQUE Constraint
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

## 3. Insert Values

### Category Table:

```
SQL> INSERT INTO Category VALUES ('Cat0001', 'Skincare', 'Facial serums, cleansers, and moisturizers');
1 row created.

SQL> INSERT INTO Category VALUES ('Cat0002', 'Makeup', 'Lipsticks, foundations, and eye cosmetics');
1 row created.

SQL> INSERT INTO Category VALUES ('Cat0003', 'Haircare', 'Shampoos, conditioners, and hair oils');
1 row created.

SQL> INSERT INTO Category VALUES ('Cat0004', 'Fragrance', 'Perfumes and body mists');
1 row created.
```

Like this, insert the remaining categories: Haircare, Fragrance.

### Product Table:

```

SQL> INSERT INTO Product VALUES ('PRD001', 'Vitamin C Serum', 'Cat0001', 899.00, 25, 'GlowLab');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD002', 'Hydrating Moisturizer', 'Cat0001', 650.00, 40, 'DermaPure');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD003', 'Sunscreen SPF 50', 'Cat0001', 499.00, 50, 'SunShield');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD004', 'Matte Liquid Lipstick', 'Cat0002', 599.00, 30, 'VelvetGlam');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD005', 'Full Coverage Foundation', 'Cat0002', 1299.00, 15, 'LuxeBeauty');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD006', 'Argan Hair Oil', 'Cat0003', 750.00, 20, 'SilkStrands');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD007', 'Repair Shampoo', 'Cat0003', 450.00, 35, 'SilkStrands');
1 row created.

SQL> INSERT INTO Product VALUES ('PRD008', 'Eau De Parfum Spray', 'Cat0004', 2499.00, 10, 'AromaFlora');
1 row created.

```

Like this, insert a few more rows.

### After Insertion - Category Table:

```

SQL> SELECT * FROM Category;

CATEGOR CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0001 Skincare
Facial serums, cleansers, and moisturizers

Cat0002 Makeup
Lipsticks, foundations, and eye cosmetics

Cat0003 Haircare
Shampoos, conditioners, and hair oils

CATEGOR CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0004 Fragrance
Perfumes and body mists

```

## After Insertion – Product Table:

```
SQL> SELECT * FROM Product;
```

| PRODUCT        | PRODUCT_NAME                              | CATEGOR | PRICE |
|----------------|-------------------------------------------|---------|-------|
| STOCK_QUANTITY | BRAND                                     |         |       |
| PRD001         | Vitamin C Serum<br>25 GlowLab             | Cat0001 | 899   |
| PRD002         | Hydrating Moisturizer<br>40 DermaPure     | Cat0001 | 650   |
| PRD003         | Sunscreen SPF 50<br>50 SunShield          | Cat0001 | 499   |
| PRD004         | Matte Liquid Lipstick<br>30 VelvetGlam    | Cat0002 | 599   |
| PRD005         | Full Coverage Foundation<br>15 LuxeBeauty | Cat0002 | 1299  |
| PRD006         | Argan Hair Oil<br>20 SilkStrands          | Cat0003 | 750   |
| PRD007         | Repair Shampoo<br>35 SilkStrands          | Cat0003 | 450   |
| PRD008         | Eau De Parfum Spray<br>10 AromaFlora      | Cat0004 | 2499  |

8 rows selected.

## CRUD Operations:

### Insert Record:

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity, Brand)
2 VALUES ('PRD009', 'Niacinamide Serum', 'Cat0001', 699.00, 18, 'GlowLab');
```

1 row created.

Update Price:

```
SQL> UPDATE Product
  2 SET Price = 550.00
  3 WHERE Product_Name = 'Sunscreen SPF 50';

1 row updated.
```

Delete a Service:

```
SQL> DELETE FROM Product
  2 WHERE Product_Name = 'Repair Shampoo';

1 row deleted.
```

## 6. Generate Category-wise Product Reports

### 1. Display all products under each category

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Brand, p.Price, p.Stock_Quantity
  2 FROM Product p
  3 JOIN Category c ON p.Category_ID = c.Category_ID;
```

| CATEGORY_NAME | PRODUCT_NAME             | BRAND      | PRICE | STOCK_QUANTITY |
|---------------|--------------------------|------------|-------|----------------|
| Skincare      | Vitamin C Serum          | GlowLab    | 899   | 25             |
| Skincare      | Hydrating Moisturizer    | DermaPure  | 650   | 40             |
| Skincare      | Sunscreen SPF 50         | SunShield  | 550   | 50             |
| Makeup        | Matte Liquid Lipstick    |            |       |                |
| Makeup        | Full Coverage Foundation | LuxeBeauty | 1299  | 15             |
| Haircare      |                          |            |       |                |

## Haircare

CATEGORY\_NAME

PRODUCT\_NAME

BRAND

PRICE

STOCK\_QUANTITY

Argan Hair Oil

SilkStrands

750

20

Fragrance

Eau De Parfum Spray

AromaFlora

2499

10

CATEGORY\_NAME

PRODUCT\_NAME

BRAND

PRICE

STOCK\_QUANTITY

Skincare

Niacinamide Serum

GlowLab

699

18

8 rows selected.

2.Count number of services in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  GROUP BY c.Category_Name;
```

CATEGORY\_NAME

PRODUCT\_COUNT

Haircare

1

Makeup

2

Skincare

4

Fragrance

1

3.Find highest-priced service in each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  WHERE p.Price = (
5    SELECT MAX(p2.Price)
6    FROM Product p2
7    WHERE p2.Category_ID = p.Category_ID
8  );
```

CATEGORY\_NAME

PRODUCT\_NAME

HIGHEST\_PRICE

Skincare

Vitamin C Serum

899

Makeup

Full Coverage Foundation

1299

Haircare

Argan Hair Oil

750

CATEGORY\_NAME

PRODUCT\_NAME

HIGHEST\_PRICE

Fragrance

Eau De Parfum Spray

2499

4.Display categories having more than 2 Products.

```
SQL> SELECT c.Category_Name
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  GROUP BY c.Category_Name
5  HAVING COUNT(p.Product_ID) > 2;

CATEGORY_NAME
-----
Skincare
```

5.Find average service price category-wise.

```
SQL> SELECT c.Category_Name
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  GROUP BY c.Category_Name
5  HAVING COUNT(p.Product_ID) > 2;

CATEGORY_NAME
-----
Skincare

SQL> SELECT c.Category_Name, ROUND(AVG(p.Price), 2) AS Avg_Price
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  GROUP BY c.Category_Name;

CATEGORY_NAME          AVG_PRICE
-----
Haircare                750
Makeup                  949
Skincare                699.5
Fragrance              2499
```